Assignment Handout: SMS Spam vs. Ham Classification Using a Convolutional Neural Network (CNN)

Deep Learning / Natural Language Processing Course

1 Assignment Overview

In this assignment, you will build, train, and evaluate a **Convolutional Neural Network (CNN)** for text classification using PyTorch. Your task is to build a binary classifier that automatically detects whether a text message is **Spam** (junk/unsolicited) or **Ham** (legitimate) using the classic SMS Spam Collection dataset.

Instead of full solutions, this guide provides structural hints, necessary PyTorch modules, and function signatures to help you implement the code yourself.

2 Objectives

1. **Data Ingestion & Preprocessing:** Load data from the web using `pandas`, map string labels (`ham`/`spam`) to binary integers (`0`/`1`), build a word-to-index vocabulary, and pad sequences to a uniform length.
2. **Build a Text CNN:** Construct a neural network utilizing an embedding layer, 1D convolutions (`nn.Conv1d`) with multiple filter sizes, max-pooling, and dropout.
3. **Train the Network:** Write an epoch-based training loop utilizing backpropagation, mini-batches, and binary cross-entropy loss.
4. **Evaluate & Test:** Track validation metrics and write an inference function to classify custom text messages.

3 Implementation Hints & PyTorch Function Guide

Part 1: Dataset & Data Loading Hints

- **Dataset Source & Downloading:** Use the pandas function `pd.read_csv()` to download and load the CSV file directly from this public web URL:
  `https://raw.githubusercontent.com/mohitgupta-101/Kaggle-SMS-Spam-Collection-Dataset-/master/spam.csv`
  (Note: You will need to pass `encoding='latin-1'` inside `pd.read_csv()` to prevent Unicode decoding errors).
- **PyTorch Tools:** Inherit from `torch.utils.data.Dataset` to create your custom dataset class, and wrap it using `torch.utils.data.DataLoader` for batching and shuffling.
- **Padding & Truncating:** Define a constant `MAX_LEN`. If a text token list is shorter than `MAX_LEN`, pad it with a special `<pad>` index. If it is longer, slice it down to `MAX_LEN`.

Part 2: Model Architecture Hints (`nn.Module`)

Your `SpamCNN` class should inherit from `nn.Module` and use the following PyTorch building blocks in its `__init__`:

- `nn.Embedding(vocab_size, embedding_dim, padding_idx=...)`: Converts word indices into dense continuous vectors. Make sure to pass `padding_idx` so the padding token remains zeroed.
- `nn.ModuleList` & `nn.Conv1d(in_channels, out_channels, kernel_size)`: Create a list of 1D convolution layers with varying `kernel_size` values (e.g., `[3, 4, 5]`) to capture n-gram phrases.
- **Dimension Transformation (`.permute`)**: Remember that `nn.Conv1d` expects the shape `[batch_size, embedding_dim, seq_len]`, whereas `nn.Embedding` outputs `[batch_size, seq_len, embedding_dim]`. Call `embedded.permute(0, 2, 1)` right after your embedding layer.
- **Pooling & Regularization**: Use `F.max_pool1d()` combined with `F.relu()` in your forward pass, followed by an `nn.Dropout()` layer before passing features into a final `nn.Linear()` classification layer.

Part 3: Training Loop Mechanics Hints

Inside your training loop over each batch, make sure to use these essential PyTorch functions and methods in order:

1. `optimizer.zero_grad()`: Clears old gradients accumulated from the previous step.
2. `loss = criterion(predictions, labels)`: Compute error using `nn.BCEWithLogitsLoss()` (which combines sigmoid and binary cross-entropy for numerical stability).
3. `loss.backward()`: Computes gradients via backpropagation.
4. `optimizer.step()`: Updates model weights using an optimizer like `optim.Adam(model.parameters(), lr=0.001)`.

Part 4: Evaluation & Inference Hints

- **Evaluation Mode**: Always toggle `model.eval()` and wrap your validation/testing loops inside a `with torch.no_grad():` context manager to save memory and disable gradient tracking.
- **Thresholding**: Pass your model's raw output logits through `torch.sigmoid()` to get probabilities between `0` and `1`. Use `torch.round()` or a threshold of `0.5` to separate **Ham (0)** from **Spam (1)**.

4 Starter Code Skeleton

Use this structural shell to guide your coding process in your notebook cells:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import pandas as pd

# 1. Load data from web URL using pd.read_csv() and preprocess...

# 2. Build Dataset and DataLoader classes...

# 3. Define CNN Architecture class...
class SpamCNN(nn.Module):
    def __init__(self, ...):
        super().__init__()
        # TODO: Define self.embedding, self.convs, self.dropout, self.
            fc

    def forward(self, text):
        # TODO: Embed -> Permute -> Conv/ReLU/MaxPool -> Concat ->
            Dropout -> Linear
        pass

# 4. Initialize model, criterion (BCEWithLogitsLoss), and optimizer (
    Adam)...

# 5. Write training & validation loops across epochs...

# 6. Write inference function to test custom messages...
```

5 Custom Test Messages

Use these test strings in your final inference cell to check if your model correctly identifies spam vs. ham:

```
custom_messages = [
    "Congratulations! You've been selected to win a free iPhone. Click
        here to claim.",
    "Hey, are we still on for our study session at the library this
        evening?",
    "URGENT: Your bank account has been locked. Verify your password
        immediately.",
    "Can you please pick up some coffee and groceries on your way home?
        "
]
```

6 Grading Rubric (Total: 5 Points)

Your assignment will be evaluated based on the following criteria:

1. **Data Ingestion & Preprocessing (1.0 Point):**

- Successfully loads the dataset from the web using `pandas` with correct encoding.
- Accurately maps labels, tokenizes text, builds a vocabulary index, and applies correct padding/truncating (`MAX_LEN`).

2. **Model Architecture Design (1.0 Point):**
   - Correctly implements the `SpamCNN` class inheriting from `nn.Module`.
   - Properly sets up `nn.Embedding` (with `padding_idx`), `nn.ModuleList` with 1D convolutions, dropout, and the final linear layer.
   - Applies the correct dimension permutation (`.permute`) between the embedding and convolution layers.
3. **Training Loop Implementation (1.0 Point):**
   - Properly sets up mini-batching using `DataLoader`.
   - Correctly calls `optimizer.zero_grad()`, computes `BCEWithLogitsLoss`, calls `loss.backward()`, and executes `optimizer.step()`.
   - Trains the model across multiple epochs showing progress.
4. **Evaluation & Validation (1.0 Point):**
   - Correctly toggles `model.eval()` and uses `torch.no_grad()` during the validation phase.
   - Computes and reports validation accuracy or loss across epochs.
5. **Inference & Custom Testing (1.0 Point):**
   - Implements a working inference function that takes raw text strings and preprocesses them.
   - Successfully tests the model on the provided custom test messages, outputting sensible predictions (Spam vs. Ham) with confidence scores.